Symbolic Task Planning: STRIPS and PDDL

Contents

1. Overview . . . 2
2. Describing the task with facts . . . 4
   - 2.1 Facts and states . . . 4
   - 2.2 Why we never list every state . . . 5
3. STRIPS actions . . . 6
   - 3.1 Precondition, add list and delete list . . . 6
   - 3.2 Applying an action . . . 7
   - 3.3 Why lists of changes: the frame problem . . . 9
4. PDDL: the task in two files . . . 11
   - 4.1 One action with parameters . . . 11
   - 4.2 The domain file . . . 12
   - 4.3 The problem file . . . 13
5. Forward search: finding the plan . . . 14
   - 5.1 The state graph, built while searching . . . 14
   - 5.2 A heuristic cuts the search . . . 17
   - 5.3 From the plan back to routes . . . 18
6. Summary . . . 20
7. Sources . . . 20
8. Key terms . . . 21

Where this fits:

- **Builds on:** Breadth-first search; Choosing the search state; Explicit and implicit graphs; A* search.

1. Overview

Key point: Carrying the cup from the table to the door takes steps in a fixed order (go to the table, pick up the cup, go to the door, put it down), not one route. We write the world as true-or-false facts and each action as the facts it needs, makes true and makes false; the task then becomes a search over sets of facts, which the graph searches of the earlier Notes already know how to do.

The robot can already plan a route. On the room's occupancy grid of 0.5 m cells, **breadth-first search** (G-2550) takes it from its start cell to the cell just inside the door in 8 moves of 0.5 m, 4.0 m in all. With diagonal moves, **Dijkstra's algorithm** (G-2565) and **A*** (G-2577) find the shortest route, 3.414 m. The robot stands at (2 m, 1 m), on the corner of its start cell.

Now give the robot a task (Figure 1): a cup stands on the table; carry it to the door. Route search fails on this task in two ways:

- its **search state** (G-2562), what the search keeps for each place it reaches, is one of the 76 free cells, so it records where the robot is and nothing about the cup or the robot's hand;
- the task needs steps in a fixed order: go to the table, pick up the cup, go to the door, put it down. The shortest route to the door, 3.414 m, leaves the cup behind.

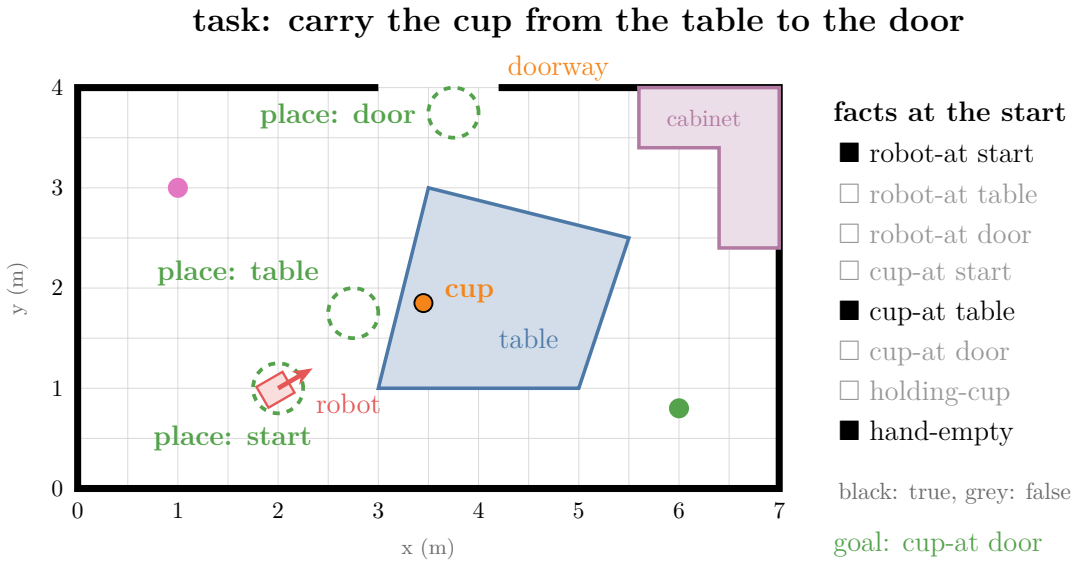


Figure 1: The room of the chapter, 7 m by 4 m, with the table, the cabinet and the two poles. The robot stands at (2, 1) facing 30 degrees; a cup stands on the table near its left edge. Three places are marked with dashed green circles: start (the robot's spot), table (beside the table, where the robot can reach the cup) and door (just inside the doorway). On the right, the eight facts of the task, with the three true at the start in black: robot-at start, cup-at table, hand-empty. The goal is cup-at door

So the robot needs two things: a way to write down where things are and what it holds, and a way to write down what each step changes. Choosing a sequence of steps from a description of the world in true-or-false statements is **symbolic task planning** (G-2600) (P&M §6.1; LaValle §2.4). "Symbolic" because the planner reasons with names, such as "the cup is on the table", not with coordinates in metres.

The planner works with three places, marked in Figure 1:

- **start**, where the robot stands now: the start cell (x 2.0 to 2.5 m, y 1.0 to 1.5 m);
- **table**, the cell beside the table's left edge (x 2.5 to 3.0 m, y 1.5 to 2.0 m), where the robot can reach the cup;
- **door**, the goal cell of route search, just inside the doorway (x 3.5 to 4.0 m, y 3.5 to 4.0 m).

Going from one place to another is one step for the planner; the route search of the earlier Notes drives it (Section 5.3). In this Note we build the planner for the cup task:

- facts and states, and why we never list every state (Section 2);
- STRIPS actions, with their precondition, add list and delete list, and the frame problem they avoid (Section 3);
- PDDL, the standard way to write a task down for any planner (Section 4);
- forward search, which finds the plan on a graph of states, first breadth-first and then with A* (Section 5).

2. Describing the task with facts

> **Key point:** The first gap of route search is that a cell says nothing about the cup. We fix it with eight true-or-false facts about the robot, the cup and the hand; a state is the set of facts true now, and the start state holds `robot-at start`, `cup-at table` and `hand-empty`.

2.1 Facts and states

A grid cell says nothing about the cup, so we list the statements about the task that can be true or false. Such a statement is a **fact** (G-2601) (P&M §6.1; LaValle §2.4.1). The cup task needs eight facts:

- `robot-at start`, `robot-at table`, `robot-at door`: where the robot is;
- `cup-at start`, `cup-at table`, `cup-at door`: where the cup is lying;
- `holding-cup`: the robot holds the cup;
- `hand-empty`: the robot holds nothing.

The list on the right of Figure 1 marks the facts that are true at the start. The set of facts that are true at one moment is a **state** (G-2602) of the task; every fact not in the set is false. The start state, which we name S0, holds three facts:

- `robot-at start`;
- `cup-at table`;
- `hand-empty`.

`hand-empty` looks redundant, since `holding-cup` being false says the same. `hand-empty` is needed because a STRIPS precondition lists only facts that must be true, never facts that must be false (FAU AI-1 notes §8.4), so "the hand is empty" must be a fact of its own; the Blocks World domain, a standard planning example, uses `arm-empty` the same way (UMBC PDDL notes).

The task asks for one fact: `cup-at door`. The set of facts that must all be true at the end is the goal of the task (P&M §6.2). The goal says nothing about where the robot ends up, so every state that holds `cup-at door` meets it: with the robot at the start, at the table or at the door (LaValle §2.4.1: a goal stands for a set of states).

A sequence of actions that turns the start state into a state that meets the goal is a plan (P&M §6.2). For the cup task a person sees one at once:

1. move from the start to the table;
2. pick up the cup;
3. move from the table to the door;
4. put the cup down.

A computer does not see the plan. The computer must be able to tell, for any state, which actions it can take and where they lead. The next question is how many states it might have to consider.

2.2 Why we never list every state

A computer could list every state with the result of every action in it. Each fact is true or false on its own, so every new fact doubles the number of possible sets of facts. Figure 2 grows the true-or-false choices for the three `robot-at` facts, one fact per frame. Watch each level split every set in two: 1 set, then 2, 4 and 8. In the frame "only 3 of the 8 sets can happen", only {start}, {table} and {door} stay, because the robot is always in exactly one place.

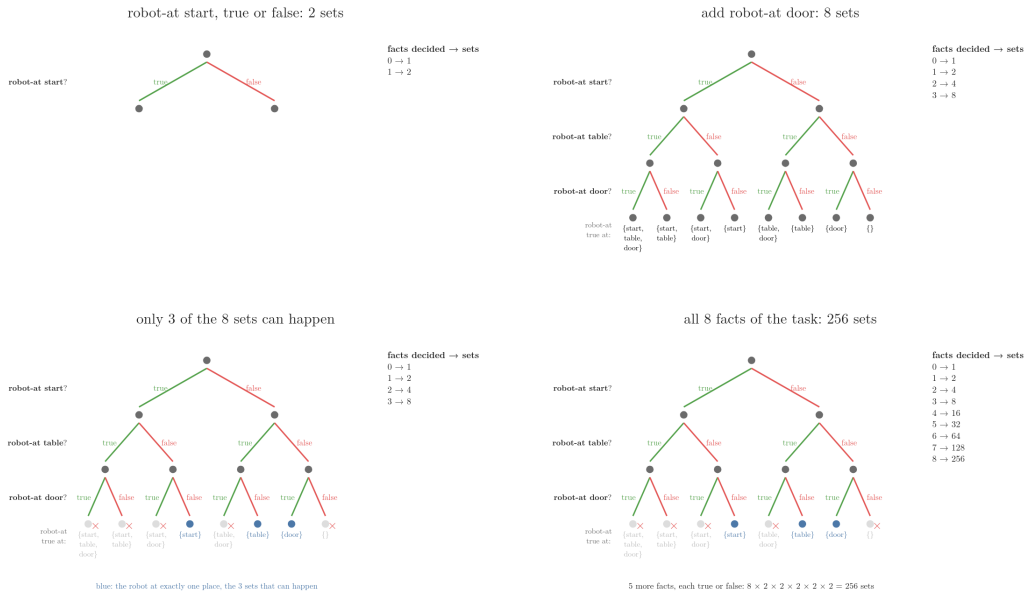


Figure 2: A tree of true-or-false choices, one fact per frame. Frame 'no fact decided yet': 1 set. Frames for robot-at start, robot-at table and robot-at door: each set splits into a true branch (green) and a false branch (red), giving 2, 4 and 8 sets; the 8 leaves list the places where robot-at is true, from {start, table, door} to {}. Frame 'only 3 of the 8 sets can happen': {start}, {table} and {door} turn blue, the other 5 are crossed out. Last frame: the count table on the right reaches 8 facts and 256 sets

With all eight facts, eight twos are multiplied:

$$2 \times 2 = 4$$

$$2 \times 2 \times 2 = 8$$

$$2^8 = 256$$

The same in symbols:

- n is the number of facts, 8 for the cup task;

- N is the number of possible sets of facts.

$$N = 2^n$$

$$= 2^8 = 256$$

Most of the 256 sets can never happen, such as a robot at the start and at the door at once. From the start state only 12 can (notebook): the robot in one of 3 places, and the cup in one of 4 spots (on the table, in the hand, at the start, at the door):

$$3 \times 4 = 12$$

Even so, a table that lists every state with the result of every action in it, such as "in S0, moving to the table gives S1", is a poor way to describe a task, for three reasons (P&M §6.1.1):

1. **Too many states.** A fourth place adds two facts, one `robot-at` and one `cup-at`, so the 8 facts become 10 and the number of possible sets grows from 256 to 1024.
2. **One change rewrites everything.** Adding a battery level to the task changes every state in the table.
3. **The pattern is hidden.** Moving from the start to the table changes only where the robot is, in every state where the move is possible, but a table of states cannot say so.

Reason 1 in numbers, with 10 facts:

$$2^{10} = 1024$$

The third reason is the way out: describe each action once, by the few facts it changes, and let the computer build the states when it needs them.

3. STRIPS actions

> **Key point:** The second need of the Overview, a way to write down what each step changes, is met by describing each action once by three lists of facts: its precondition (what must be true to do it), its add list (what it makes true) and its delete list (what it makes false). Every fact on neither list keeps its value; in the state S1, `pick-up table` removes `cup-at table` and `hand-empty`, adds `holding-cup` and leaves the other five facts alone.

3.1 Precondition, add list and delete list

Section 2.2 left us needing a description of each action by the few facts it changes. Take picking up the cup at the table:

- before it, three facts must be true: the robot is at the table, the cup is on the table, the hand is empty;
- after it, one fact has become true: the robot holds the cup;
- and two facts have become false: the cup is no longer on the table, and the hand is no longer empty.

These three lists are how the STRIPS planner describes an action (P&M §6.1.2; LaValle §2.4.1). STRIPS, the STanford Research Institute Problem Solver, was the planner of Shakey, one of the first robots built with artificial intelligence (P&M §6.1.2). The three lists are:

- the **precondition** (G-2606): the facts that must be true for the action to be done;
- the **add list** (G-2607): the facts the action makes true;
- the **delete list** (G-2608): the facts the action makes false.

An action described by these three lists is a **STRIPS action** (G-2605). The four actions of the plan of Section 2.1:

Action	Precondition	Add list	Delete list
`move start table`	`robot-at start`	`robot-at table`	`robot-at start`
`pick-up table`	`robot-at table`, `cup-at table`, `hand-empty`	`holding-cup`	`cup-at table`, `hand-empty`
`move table door`	`robot-at table`	`robot-at door`	`robot-at table`
`put-down door`	`robot-at door`, `holding-cup`	`cup-at door`, `hand-empty`	`holding-cup`

A move deletes the old place because the robot cannot be in two places at once. The other actions of the task are the same three kinds at the other places (notebook):

- a move from each of the 3 places to each of the other 2;
- a pick-up at each of the 3 places;
- a put-down at each of the 3 places.

$$3 \times 2 = 6 \text{ moves}$$

$$6 + 3 + 3 = 12 \text{ actions}$$

The 12 actions describe the whole task. What they are for is applying them to a state, which Section 3.2 does.

3.2 Applying an action

With the three lists, the computer can now check the plan of Section 2.1 by itself, one action at a time. An action can be done in a state only when every fact of its precondition is true

there; the action is then applicable in that state (P&M §6.2). Doing it takes the delete list out of the state and puts the add list in (P&M §6.1.2; LaValle §2.4.1). Our example is `pick-up table` in the state S1, the state after `move start table`:

- s is the state before the action, a set of facts; here S1, which holds `robot-at table`, `cup-at table` and `hand-empty`;
- pre is the action's precondition: `robot-at table`, `cup-at table`, `hand-empty`;
- del is its delete list: `cup-at table`, `hand-empty`;
- add is its add list: `holding-cup`;
- s' is the state after the action;
- $\subseteq$ means "every fact on the left is also on the right": the set {`robot-at table`} $\subseteq$ S1 is true, while {`holding-cup`} $\subseteq$ S1 is false;
- $\setminus$ means "take out": S1 $\setminus$ del keeps the facts of S1 that are not in del;
- $\cup$ is the **union** (G-2045) of two sets: every fact that is in either one.

The test for applying the action, and the next state:

$$\text{pre} \subseteq s$$

$$s' = (s \setminus \text{del}) \cup \text{add}$$

On S1, the three facts of the precondition are all in S1, so `pick-up table` is applicable. Taking out the delete list leaves one fact:

$$s \setminus \text{del} = \{\text{robot-at table}\}$$

Putting in the add list gives the next state. We call it S3, because Section 5.1 numbers the states in the order its search makes them:

$$s' = \{\text{robot-at table}, \text{holding-cup}\}$$

The order of the two steps does not matter here, because no fact is on both lists of an action.

Figure 3 applies the whole plan this way, one action at a time. Watch the fact list on the right. In each frame named "is its precondition true?", the needed facts turn blue. In the next frame, named "delete list removed, add list added", the deleted facts turn red and the added ones green, while the room on the left shows the robot and the cup where the facts now say they are. The last frame, "goal reached", holds `cup-at door`, so the plan meets the goal.

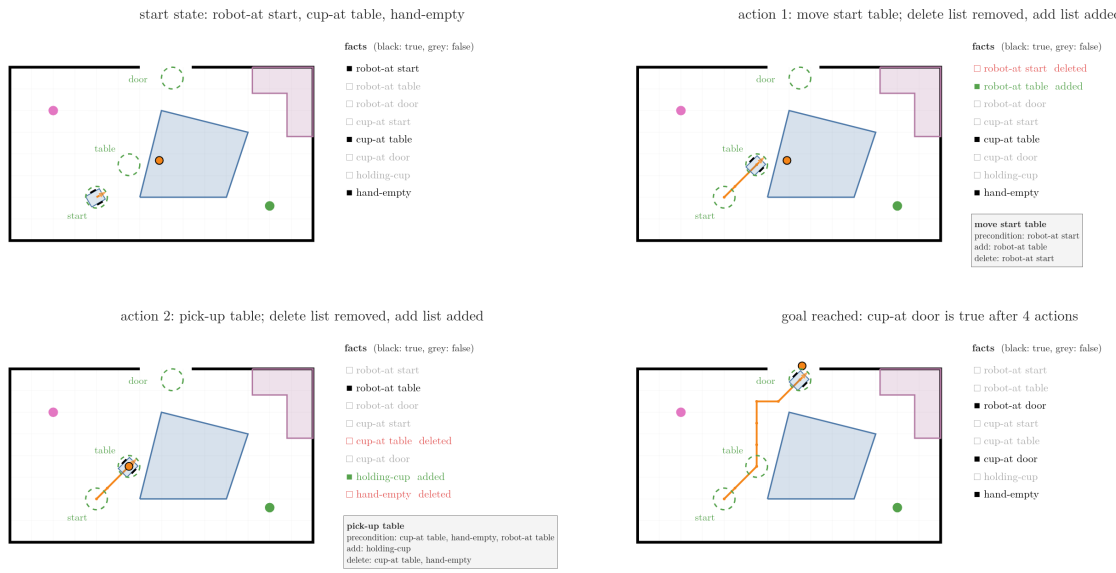


Figure 3: The four-action plan applied, one action per stage. Left: the room, with the robot driving each move along a shortest route of grid cells (orange trail) and the cup (orange dot) on the table, then in the robot's hand, then at the door. Right: the eight facts, black when true and grey when false, with a card giving the action's precondition, add list and delete list. In the stage 'is its precondition true?' the precondition's facts are blue with a tick; in the stage 'delete list removed, add list added' the deleted facts are red and the added ones green. The last frame shows robot-at door, cup-at door and hand-empty true: goal reached after 4 actions

Checking a plan we were given takes four applications of the rule. Two questions are still open: why describe actions by their changes at all (Section 3.3), and how a computer finds a plan that nobody gave it (Section 5).

3.3 Why lists of changes: the frame problem

Section 3.2 changed only 3 of the 8 facts with `pick-up table`; the other 5 stayed as they were. Writing down what every fact is after the action needs a line for each of the 8 facts, and 5 of those 8 lines only say "stays the same". Figure 4 puts the two descriptions of `pick-up table` side by side: on the left, the full list, with five grey lines that say "stays"; on the right, the STRIPS lists, with only the three coloured changes.

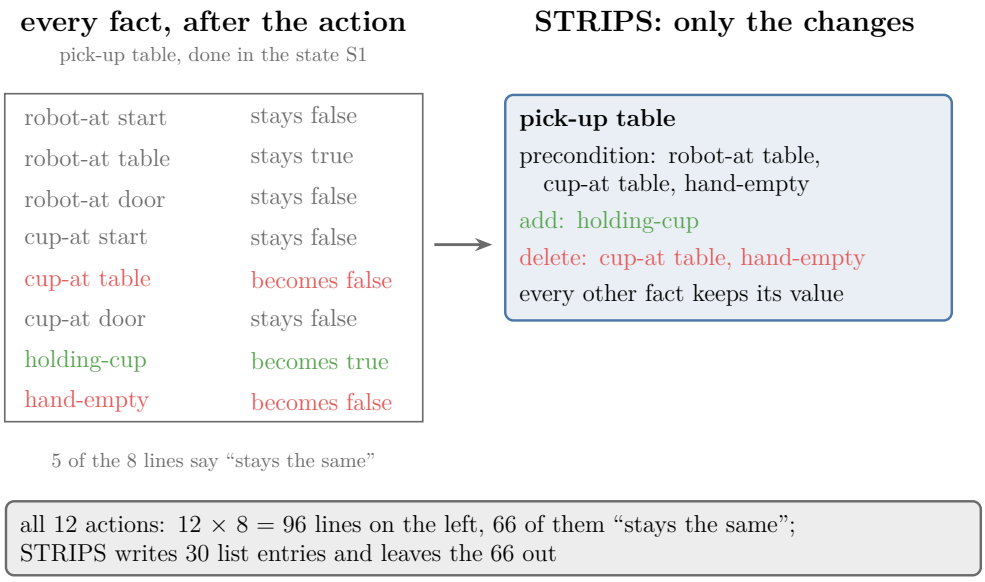


Figure 4: The action pick-up table, done in the state S1, described two ways. Left, 'every fact, after the action': eight lines; robot-at start, robot-at table, robot-at door, cup-at start and cup-at door are grey and say 'stays false' or 'stays true'; cup-at table and hand-empty are red, 'becomes false'; holding-cup is green, 'becomes true'. Right, 'STRIPS: only the changes': precondition robot-at table, cup-at table, hand-empty; add holding-cup; delete cup-at table, hand-empty; every other fact keeps its value. Below: over all 12 actions, 12 times 8 = 96 lines on the left, 66 of them 'stays the same'; STRIPS writes 30 list entries and leaves the 66 out

Over all 12 actions of the task:

- A is the number of actions, 12;
- n is the number of facts, 8;
- c is the number of changes, counted over all actions: 2 for each move (one fact added, one deleted), 3 for each pick-up and each put-down;
- L is the number of lines needed to say every fact after every action;
- F is the number of those lines that only say "stays the same".

The changes first, counted from the action lists:

$$c = 6 \times 2 + 3 \times 3 + 3 \times 3$$

$$= 12 + 9 + 9 = 30$$

Then the lines, our numbers first, each followed by the same step in symbols:

$$L = 12 \times 8 = 96$$

$$L = A\,n$$

$$F = 96 - 30 = 66$$

$$F = A\,n - c$$

So 66 of the 96 lines say only that nothing else changes. Written in logic, each of these "stays the same" statements must be spelled out, one for each action and each fact it leaves alone, and their number grows with every new fact and every new action (P&M §6.1.3 calls them frame rules). Having to state everything an action does not change is the **frame problem** (G-2610) (FAU AI-1 notes §8.1). An everyday comparison: a recipe says "add the salt"; it never adds "the flour, the oven and the rest of the kitchen stay as they are".

STRIPS avoids the 66 lines with one rule: a fact that is on neither list keeps its value (P&M §6.1.2). So STRIPS writes the 30 changes and nothing else, which is why a state can be updated by taking out the delete list and putting in the add list, as in Section 3.2.

The price is that every STRIPS action has fixed lists. A move to the table and a move to the door change different facts, so each pair of places needs its own move action (P&M §6.1.2). With 3 places that is 6 moves; Section 4 shows how the count grows, and how a planning language writes all of them at once.

4. PDDL: the task in two files

> **Key point:** To hand the cup task to a planner, we write it in PDDL, the standard language that planners read. STRIPS needs one move action per pair of places, 6 for our 3 places and 90 for 10; PDDL writes one move with the parameters ?from and ?to, and splits the task into a domain file (the predicates and actions, written once) and a problem file (the places, the start state and the goal, one per task).

4.1 One action with parameters

Section 3.3 ended with one STRIPS move action per pair of places. A move leaves one of the places for one of the others:

- k is the number of places, 3 for the cup task;
- M is the number of STRIPS move actions.

$$M = 3 \times 2 = 6$$

$$M = k\,(k - 1)$$

A building with 10 places would need:

$$M = 10 \times 9 = 90$$

All 90 say the same thing: be at one place, end at another. So we write the move once, with blanks for the two places, and let the planner fill them in. Such a blank is a **parameter** (G-2614) of the action, written with a question mark: `move ?from ?to` (UMBC PDDL notes; LaValle §2.4.1). Facts get blanks too: `robot-at ?p` stands for the three facts `robot-at start`, `robot-at table` and `robot-at door`. A fact pattern with blanks is a **predicate** (G-2613) (LaValle §2.4.1).

The language that writes tasks this way is the **Planning Domain Definition Language** (G-2612), PDDL, defined in 1998 for the International Planning Competition and read by many planners today (planning.wiki; UMBC PDDL notes). A common language means a robot's task can be handed to any planner, and a better planner can replace the old one without rewriting the task. PDDL keeps the STRIPS idea: an action's effect lists the facts it makes true, and the facts it makes false inside `(not ...)`. A PDDL task comes in two files, one for what never changes and one for the task itself, which Sections 4.2 and 4.3 write for the cup task.

4.2 The domain file

The parts of the cup task that never change go in the **domain file** (G-2615): the predicates and the three actions (UMBC PDDL notes). Our domain file, `data/cup-domain.pddl`:

```
(define (domain cup-room)
  (:requirements :strips)
  (:predicates (robot-at ?p) (cup-at ?p) (holding-cup) (hand-empty))

  (:action move
    :parameters (?from ?to)
    :precondition (robot-at ?from)
    :effect (and (robot-at ?to) (not (robot-at ?from))))

  (:action pick-up
    :parameters (?p)
    :precondition (and (robot-at ?p) (cup-at ?p) (hand-empty))
    :effect (and (holding-cup) (not (cup-at ?p)) (not (hand-empty))))

  (:action put-down
    :parameters (?p)
    :precondition (and (robot-at ?p) (holding-cup))
    :effect (and (cup-at ?p) (hand-empty) (not (holding-cup)))))
```

Reading it line by line:

- `:requirements :strips` says that the file uses only the STRIPS features of PDDL: preconditions, add lists and delete lists;
- `:predicates` lists the four predicates;
- each `:action` has its `:parameters`, its `:precondition`, and an `:effect` that holds the add list (the plain facts) and the delete list (the facts inside `not`).

The `move` action with `?from` = start and `?to` = table is the STRIPS action `move start table` of Section 3.1, with the same three lists. The many round brackets are the notation of the Lisp programming language, which PDDL keeps (UMBC PDDL notes). The domain file says what the robot can do, but not where it starts or what it must achieve.

4.3 The problem file

What changes from task to task goes in the **problem file** (G-2616): the objects (here the three places), the start state and the goal (UMBC PDDL notes). Our problem file, `data/cup-problem.pddl`:

```
(define (problem cup-to-door)
  (:domain cup-room)
  (:objects start table door)
  (:init (robot-at start) (cup-at table) (hand-empty))
  (:goal (cup-at door)))
```

`:init` is the start state S0 of Section 2.1, and `:goal` the goal. Figure 5 shows how the files are used: the planner reads both and returns a plan. A new task, such as bringing the cup back from the door to the table, needs only a new problem file with a new `:init` and `:goal` (in `data/cup-problem-back.pddl`); the domain file stays as it is.

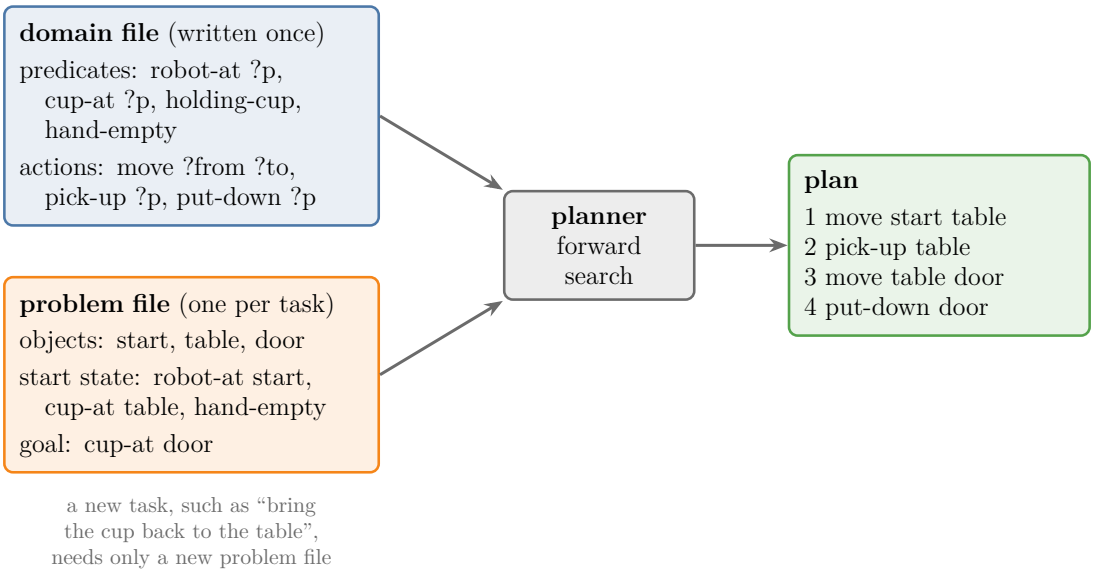


Figure 5: The domain file (written once: the predicates robot-at ?p, cup-at ?p, holding-cup, hand-empty and the actions move ?from ?to, pick-up ?p, put-down ?p) and the problem file (one per task: the objects start, table, door, the start state and the goal cup-at door) both go into the planner, which runs forward search and returns the plan: 1 move start table, 2 pick-up table, 3 move table door, 4 put-down door. A note under the problem file: a new task needs only a new problem file

As a check, the notebook reads the two files the way a PDDL planner does: it fills each action's parameters with the three places and runs breadth-first search (Section 5.1). The search returns the four actions of Section 2.1, and for the cup-back problem file it returns `move start door`, `pick-up door`, `move door table`, `put-down table`. The files say what the task is; how the planner finds the plan is the subject of Section 5.

5. Forward search: finding the plan

> **Key point:** The last part of the problem is finding the plan without being told it. Forward search starts from the start state, applies every applicable action to make the next states, and searches the resulting graph of states for one that meets the goal; breadth-first search finds the 4-action plan after taking 8 states from the frontier, and A* with a count of the actions still needed takes only 5.

5.1 The state graph, built while searching

The rule of Section 3.2 turns one state into the next. Applied to every applicable action of every state, it links the states into a **graph** (G-2540), points joined by links: each state is a **vertex** (G-2541) of the graph, and each action is an **edge** (G-2542), a link from one state to the state it leads to (P&M §6.2). The graph is never stored in full; the planner makes a

state's neighbours from the action lists only when it reaches that state, as for the **implicit graph** (G-2561) of route search, a graph whose neighbours are made on demand.

A planner that starts at the start state and moves forward, action by action, until it reaches a state that meets the goal is doing **forward search** (G-2617), also called progression (P&M §6.2; LaValle §2.4). Any graph search of the earlier Notes can run it. We start with breadth-first search (G-2550), which keeps a **frontier** (G-2547), the states made but not yet taken, and takes them in the order they were made.

From S0, two actions are applicable: `move start table` and `move start door`. `pick-up start` is not, because the cup is not at the start. So S0 has two neighbours, and the search grows from there. Figure 6 runs breadth-first search on the cup task, one state taken per frame. Columns are where the robot is and rows where the cup is; every line is a move along a row or a pick-up or put-down down a column, and each line stands for two actions, one each way. The states are numbered in the order the search makes them. Watch the frontier list at the bottom:

- frame "take S0" makes S1 and S2;
- frame "take S2: makes nothing new": with the robot at the door and the cup on the table, both moves lead back to states already made;
- S3 is the state of Section 3.2, the cup in the hand at the table;
- frame "take S7" stops, because S7 holds `cup-at door`.

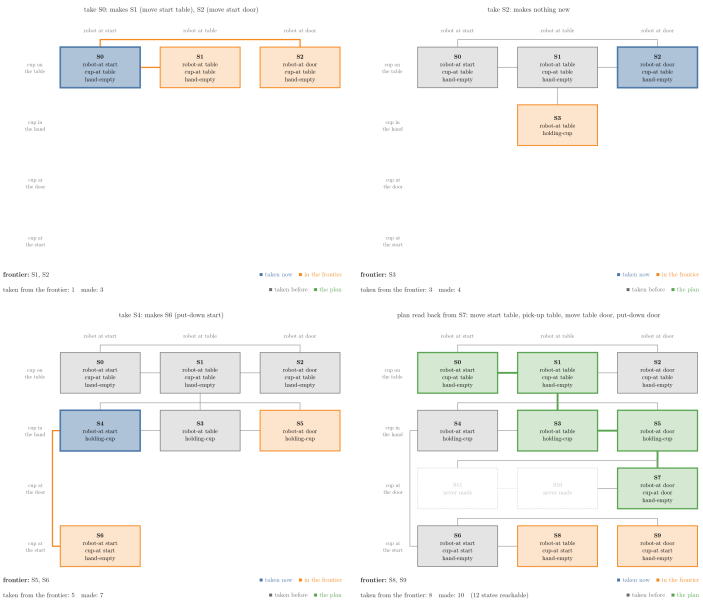


Figure 6: Breadth-first forward search on the cup task's states, one state taken per frame. Each box is a state with its true facts; columns: robot at start, table, door; rows: cup on the table, in the hand, at the door, at the start. Blue: the state taken now; orange: states in the frontier; grey: states taken before. The first frame takes S0 and makes S1 (move start table) and S2 (move start door). 'take S2: makes nothing new'. The search goes on to S3 (pick-up table), S4 and S5 (the two moves holding the cup), S6 (put-down start), S7 (put-down door), S8 and S9. Frame 'take S7' stops at the goal. Last frame: the plan read back from S7 in green, S0, S1, S3, S5, S7; S10 and S11, the other two states with the cup at the door, were never made; 8 states taken, 10 made, 12 reachable

The last frame reads the plan back from S7, following each state to the state it was made from:

1. `move start table` (S0 to S1);
2. `pick-up table` (S1 to S3);
3. `move table door` (S3 to S5);
4. `put-down door` (S5 to S7).

The search took 8 states from the frontier and made 10 of the 12 states the task can reach, the states that some sequence of actions leads to from the start (notebook). The other 244 of the 256 possible sets were never touched. Breadth-first search also returns a plan with the fewest actions, because it looks at all states one action away before any state two actions away. The plan is the one of Section 2.1, now found by the computer.

Eight states is cheap here, but the count grows with the number of facts. With n facts there can be up to 2^n states (Section 2.2), so a task with a few more places, cups and doors has far more states for breadth-first search to take, and the search needs a guide.

5.2 A heuristic cuts the search

Section 5.1 took 8 states to find a 4-action plan. A* (G-2577) takes fewer when it has a **heuristic** (G-2570), a guess of the cost still to go, that never overestimates that cost: an **admissible heuristic** (G-2578). With every action costing 1, a good guess is the fewest actions still needed, counted like the coffee-delivery example of P&M §6.2:

- if the cup is at the door: 0;
- if the robot holds the cup: 1 for the put-down, plus 1 for a move if the robot is not at the door;
- if the cup lies elsewhere: 3 for pick-up, move and put-down, plus 1 for a move to the cup if the robot is not there.

Each action counted must happen at least once, so the count never overestimates. For the start state S0:

- g is the cost so far (G-2574), here the number of actions taken, 0 in S0;
- h is the heuristic, the fewest actions still needed;
- f is their sum (G-2575), the cost A* expects for a plan through the state.

$$h = 1 + 1 + 1 + 1 = 4$$

$$f = g + h = 0 + 4 = 4$$

The four 1s are the move to the table, the pick-up, the move to the door and the put-down. Figure 7 runs A* on the same state graph, with the same state names as Figure 6, and writes g + h = f in the corner of each box. A* always takes the frontier state with the smallest f. Watch the frontier list: S2 (f = 5, the robot drove to the door without the cup) and S4 (f = 5, the robot carried the cup back to the start) wait in the frontier, while every state with f = 4 lies on the plan.

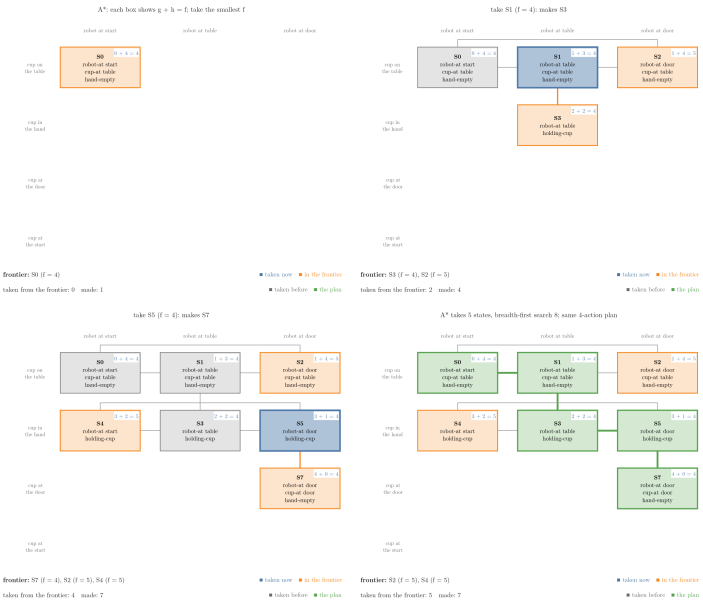


Figure 7: A* forward search on the same states as Figure 6, one state taken per frame, with g + h = f in the corner of each box (g: actions so far, h: fewest actions still needed). The first frame shows S0 with 0 + 4 = 4. 'take S1 (f = 4): makes S3' (2 + 2 = 4). 'take S5 (f = 4): makes S7' (4 + 0 = 4). Last frame: A* takes 5 states, breadth-first search 8; the same 4-action plan in green; S2 (1 + 4 = 5) and S4 (3 + 2 = 5) were made but never taken

A* takes 5 states, S0, S1, S3, S5 and S7, against 8 for breadth-first search, and returns the same 4-action plan (notebook). Our heuristic was written by hand for this one task. Planners built for any task guide forward search with a heuristic too: the default planner of the free planning.domains service is a forward search guided by a heuristic (UMBC PDDL notes). The plan is found; each of its moves still has to be driven.

5.3 From the plan back to routes

The plan of Section 5.2 says "move start table", not how to drive there. Each move is handed to the route search of the earlier Notes, on the **8-connected grid** (G-2564), the 0.5 m grid with diagonal moves, from the cell of one place to the cell of the next (P&M §6.1 leaves the actions to a lower-level controller in the same way). The last frame of Figure 3 shows the two routes:

- `move start table`: one diagonal step, 0.707 m;
- `move table door`: 2.707 m.

$$0.707 + 2.707 = 3.414 \text{ m}$$

Fetching the cup costs no extra distance, because the table place lies on a shortest route from the start to the door: the plan's two routes add up to the 3.414 m of the direct route of the

Overview (notebook). The task now has its answer: four actions in a fixed order, found by forward search, each move driven by route search.

6. Summary

Idea	What it is	Why
Fact, state	a true-or-false statement; the set of facts true now	a state can hold the cup and the hand, which a grid cell cannot
STRIPS action	precondition, add list, delete list	one description per action, not one table entry per state
Unlisted facts keep their value	a fact on neither list stays as it was	avoids the frame problem: 30 list entries instead of 96 lines
PDDL	domain file plus problem file, actions with parameters	one move instead of 6 (or 90), and any planner can read the task
Forward search	apply every applicable action from the start, search the state graph	finds the plan; A* with a heuristic takes 5 states instead of 8

- The cup task is solved by four actions in a fixed order, `move start table`, `pick-up table`, `move table door`, `put-down door`, which breadth-first forward search finds after taking 8 states, because each state records the cup and the hand that a grid cell leaves out; the two moves are routes of 0.707 m and 2.707 m, the same 3.414 m as the direct route to the door.
- A **state** (G-2602) is the set of true **facts** (G-2601); 8 facts allow 256 sets but only 12 can happen, so we never list states and let the planner make them as it goes.
- A **STRIPS action** (G-2605) lists its **precondition** (G-2606), **add list** (G-2607) and **delete list** (G-2608), so applying it is "take out the delete list, put in the add list", as `pick-up table` turns S1 into S3.
- STRIPS keeps every fact that no list mentions as it was, so the 66 "stays the same" lines of the **frame problem** (G-2610) are never written.
- **PDDL** (G-2612) writes each action once with **parameters** (G-2614), and splits the task into a **domain file** (G-2615) and a **problem file** (G-2616), so a new task needs only a new problem file and any planner can solve it.
- **Forward search** (G-2617) is graph search on states made from the action lists; breadth-first search returns the plan with the fewest actions, and A* with an admissible heuristic returns the same plan after taking 5 states instead of 8, because its guess steers it along the plan.

The robot can now turn a task with objects into an ordered list of actions, and each move into a route of grid cells. Still missing: the route moves a point from cell to cell, while the real robot is a body with a size and a heading. Configuration space describes every placement of the whole robot with a few numbers.

7. Sources

Built from

- FAU Erlangen-Nürnberg (FAU.tv), "Artificial Intelligence (AI-1), WS 2020: 3 - The STRIPS Planning Formalism", video lecture, https://www.fau.tv/clip/id/26896
- FAU Erlangen-Nürnberg (FAU.tv), "Artificial Intelligence (AI-1), WS 2020: 5 - The PDDL Language", video lecture, https://www.fau.tv/clip/id/26897
- Poole, D. L. and Mackworth, A. K. (2023). *Artificial Intelligence: Foundations of Computational Agents*, 3rd ed. Cambridge University Press. §6.1 "Representing States, Actions, and Goals", §6.2 "Forward Planning". Free online: https://artint.info/3e/html/ArtInt3e.Ch6.S1.html (P&M)
- UMBC, CMSC 471 (Spring 2022), "PDDL and the Blocks World", lecture notes, https://courses.cs.umbc.edu/471/spring22/02/notes/13_planning/13.2_pddl.pdf (UMBC PDDL notes)

Other references

- LaValle, S. M. (2006). *Planning Algorithms*. Cambridge University Press. §2.4 "Using Logic to Formulate Discrete Planning", §2.4.1 "A STRIPS-Like Representation". Free online: https://lavalle.pl/planning/node59.html (LaValle)
- planning.wiki, "What is Planning Domain Definition Language (PDDL)?", https://planning.wiki/guide/whatis/pddl (planning.wiki)
- FAU Erlangen-Nürnberg, "Artificial Intelligence 1, Lecture Notes, Part IV: Planning and Acting" (Winter Semester 2024/25), §8.1 "Logic-Based Planning", Definition 8.1.11 (the frame problem), §8.4 "The STRIPS Planning Formalism" (preconditions are facts that must be true), https://kwarc.info/teaching/AI/notes-part4.pdf (FAU AI-1 notes)

8. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
Symbolic task planning (G-2600)	Choosing a sequence of actions that reaches a goal, from a description of the world in true-or-false facts such as "the cup is on the table"; it decides the order of steps (go to the table, pick up, go to the door, put down) that a route search alone cannot.
Fact (task planning) (G-2601)	A statement about the task that is either true or false, such as `robot-at table` or `holding-cup`; facts are what a planner's states, actions and goals are made of.
State (task planning) (G-2602)	The set of facts that are true at one moment, every other fact being false, such as {`robot-at start`, `cup-at table`, `hand-empty`}; it records the objects and the robot's hand, which a grid cell cannot.

Term	Meaning
Precondition (G-2606)	The facts that must be true for an action to be done, such as `robot-at table`, `cup-at table` and `hand-empty` for picking up the cup; the planner tries an action only in states that hold all of them.
Add list (G-2607)	The facts an action makes true, such as `holding-cup` for picking up the cup; applying the action puts them into the state.
Delete list (G-2608)	The facts an action makes false, such as `cup-at table` and `hand-empty` for picking up the cup; applying the action takes them out of the state.
STRIPS action (G-2605)	An action described by three lists of facts, its precondition, add list and delete list, as in the STRIPS planner of the Shakey robot; one such description replaces a table of the action's result in every state.
Frame problem (G-2610)	The difficulty that a logical description of an action must also state everything the action leaves unchanged, one statement per action and fact (66 of 96 lines in the cup task); STRIPS avoids it with the rule that a fact on neither list keeps its value.
Parameter (PDDL action) (G-2614)	A blank in an action, written with a question mark such as `?from` and `?to` in `move ?from ?to`, that the planner fills with objects; one action with parameters replaces many STRIPS actions (6 moves for 3 places, 90 for 10).
Predicate (PDDL) (G-2613)	A fact pattern with blanks, such as `robot-at ?p`, which stands for one fact per object (`robot-at start`, `robot-at table`, `robot-at door`); it lets a domain name a whole family of facts at once.
PDDL (Planning Domain Definition Language) (G-2612)	The standard language for writing a planning task, defined in 1998 for the International Planning Competition: actions with parameters, preconditions and effects, in a domain file and a problem file; a task written in it can be handed to any planner that reads PDDL.

Term	Meaning
Domain file (PDDL) (G-2615)	The PDDL file that holds the parts of a task family that never change: its predicates and its actions; it is written once and reused for every problem in that family.
Problem file (PDDL) (G-2616)	The PDDL file that holds one task: its objects, its start state and its goal; a new task needs only a new problem file for the same domain file.
Forward search (progression) (G-2617)	Planning by starting at the start state, applying every applicable action to make the next states, and searching the resulting graph of states until one meets the goal; any graph search, such as breadth-first search or A*, can run it.
Breadth-first search (BFS) (G-2550)	A search whose frontier is a queue, so it expands vertices in rings of equal moves from the start; the first route it finds has the fewest moves, which is the shortest only when every move costs the same.
Dijkstra's algorithm (G-2565)	A search that always settles the vertex with the smallest distance from the start found so far, which is then final, and offers each of its neighbours the route through it, keeping the shorter; with every edge cost at least 0 it finds the shortest route to every vertex (3.414 m to the door on the room grid with diagonals).
A* search (G-2577)	Dijkstra's algorithm with the open list sorted by $f = g + h$, the cost so far plus a heuristic guess of the cost left; with a consistent heuristic it returns the shortest route while expanding far fewer states (9 cells against 54 on the room grid).
Search state (G-2562)	What a search keeps for each vertex: the quantities that decide which moves are allowed and what they cost, and nothing more, such as (cell, charge left) for a robot with a battery; leaving one out breaks the Markov property and can hide routes.
Union ($A \cup B$) (G-2045)	The set of elements in A or B (or both), such as the points inside either of two convex pieces of an obstacle; for events, the event that A or B (or both) happens.

Term	Meaning
Graph (G-2540)	A set of vertices (points) joined by edges (lines), such as the room's 76 free cells joined by 114 moves; it keeps only which places connect directly, so one search method works for maps, puzzles and any problem of states and moves.
Vertex (graph) (G-2541)	One point of a graph, such as one free cell of the room or one waypoint; the places a search moves between.
Edge (graph) (G-2542)	A link between two vertices of a graph, such as one 0.5 m move between two free cells that share a side; it says the search can go directly from one vertex to the other.
Implicit graph (G-2561)	A graph never built in full: the search starts from the start vertex and calls a function that returns a vertex's neighbours when it is expanded, so only the vertices the search reaches are made; this is what makes state spaces of millions of states searchable.
Frontier (open list) (G-2547)	The vertices a search has reached but not yet expanded, waiting to be taken off one at a time; which one comes off next is what makes one search differ from another.
Heuristic h (G-2570)	A cheap guess of the cost still to go from a state to the goal, such as 3.121 m from the start cell to the door; a search uses it to prefer states that look nearer the goal.
Admissible heuristic (G-2578)	A heuristic that is never more than the true cost left from any state, $h \leq h^*$; it makes f a lower bound on every route through a state, so the first route A* takes off the open list is the shortest.
Cost so far g (g-value) (G-2574)	The length of the best route found so far from the start to a state; A* lowers it whenever a shorter way into the state turns up.
Estimated total cost f (f-value) (G-2575)	The cost so far plus the heuristic, $f = g + h$: the length of the best route through a state if the guess were exact; A* expands the open state with the smallest f.

Term	Meaning
8-connected grid (G-2564)	A grid made into a graph by joining each cell to all 8 cells around it: 4 straight moves of one cell side and 4 diagonal moves of 1.414 cell sides (0.707 m for 0.5 m cells), usually without cutting the corner of a blocked cell; it gives shorter, less jagged routes than a 4-connected grid.